Design Document

Group 13: Amr Binmubarak, Sam Eubanks, Zachary Ferguson, Scott Ruona, Jit Shrestha

Sponsor: Alexandria City – Office of Analytics, Innovation, and Data

Table of Contents

1 Architectural Components 2

1.1 Presentation Layer 2
1.1.1 Client-Side Web Application 2
1.1.2 Server-Side Web Application 3
1.2 Application Layer 3
1.3 Data Layer 4

2 Resources 5

3 High-Level Architecture Design 7

4 Design Sketches 8

5 AI Usage Statement 8

6 Attachments 9

6.1 Wireframes 9
6.2 Entity Relationship Diagram 13

1 Architectural Components

The overall solution will be comprised of a layered architecture.

1. **Presentation Layer**
   This layer is responsible for user interactions and communicating with the Application Layer. It will be comprised of a web application with its own client-server architecture.
2. **Application Layer**
   This layer owns business logic and execution of software functions, and interfaces with both the presentation layer and the data layer.
3. **Data Layer**
   This layer is responsible for all data storage, including application data (e.g., users, configuration) and audit report data.

1.1 Presentation Layer

The presentation tier will be made up of a web application, using a client-server architecture.

Authenticated administrators can add new reports, edit or remove existing reports, modify user accounts, and access other advanced functions and configurations. Regular authenticated users have access limited to viewing reports and analytics dashboards.

1.1.1 Client-Side Web Application

The client-side web application will be built using web standards and proven libraries, including React.js, Typescript, and Tailwind.

Additional front-end libraries will support the development of an interactive user interface, including:

- The shadcn React component library
- TanStack Table for displaying tabular data in a shadcn component
- Recharts for displaying charts within shadcn components

The client-side application will only interface with the server-side web application through a REST API.

1.1.2 Server-Side Web Application

The web applications server-side component will route requests from the client side to the appropriate Application Layer services through a REST API and return the response to the client-side application if applicable.

It will use Flask, a popular and well-supported web backend framework that integrates well with React.js.

1.2 Application Layer

The application layer contains business logic and execution that bridges the presentation layer and the Data layer.

This layer is comprised of several microservices:

- **Scheduler**: Triggers scheduled scans of registered audit sources URLs at the frequency by the user
- **Scraper**: Fetches registered source pages and identifies newly posted audit reports by comparing URLs and checksums against the Data layer
- **PDF Text Extraction**: Converts each new PDF into structured text, with an OCR fallback for scanned documents.
- **Analytic Model**: Sends the extracted text and the predefined list to an external LLM API, and returns applicable tags with a supporting quote from the source for each tag
- **Validation:** Confirms every returned tag is in the predefined list and that each supporting quote exists in the source text before persisting.
- **User Authentication and Management:** Handles login, sessions, and role-based access.

Audit Report Processing Workflow

The workflow begins when a user submits a report URL or the Scheduler initiates a scan. The Scraper retrieves new or updated reports, and the PDF Text Extraction component converts them into machine-readable text. The Analysis Engine uses an external LLM API to generate tags, supporting quotations, and summaries. After validation, the results are stored in the MySQL database and made available to users through the web application.

1.3 Data Layer

The data layer is responsible for persisting in storage of audit report metadata in addition to further insights such as summaries or tags. Due to the goal of product containerization, the data will be stored using MySQL, a very light and popular relational database management system (RDBMS). The data layer will consist of several tables:

1. **User Authorization**: Holds user login information for confirmation of application access, pivotal to providing a secure product. The application layer will send a request to ensure credentials are valid and verify permissions. If a match is found, then the application layer will allow the user to log in.

2. **Audit Logs**: Serves as a header table containing all the immediately necessary information of an audit report. This will include an audit_id (PK), title, auditing office, date, and link to the original audit source. Storing a link avoids the burden of document storage, which would quickly clog up space.

3. **Audit Summaries**: This will only contain a summary_id (PK), audit_id (FK), and the summary. By separating the summary, you can look up tags or filter dates without the cost of sending additional memory of the summaries.

4. **Audit Tags**: This table contains a tag_id (PK), audit_id (FK), and a struct containing the tags. The struct will further be broken down into categorized tags such as geographic, department, content, etc. Each category will have a list of strings containing specific tags such as [‘Arlington’, ‘Alexandria’].

5. **Schedules**: This table will hold the timing for scanning links to audit sources. Entries will be initiated by the user and on a fixed schedule it will read through the specified link to identify new entries and create appropriate logs using the web audit ingesting process.

The goal of separating tables is to speed up lookup times for the application, avoiding unnecessary overhead pulling tags/summaries when you are only interested in the link to a specific audit.

The preliminary Entity Relationship Diagram for our database is available in the Attachments section.

2 Resources

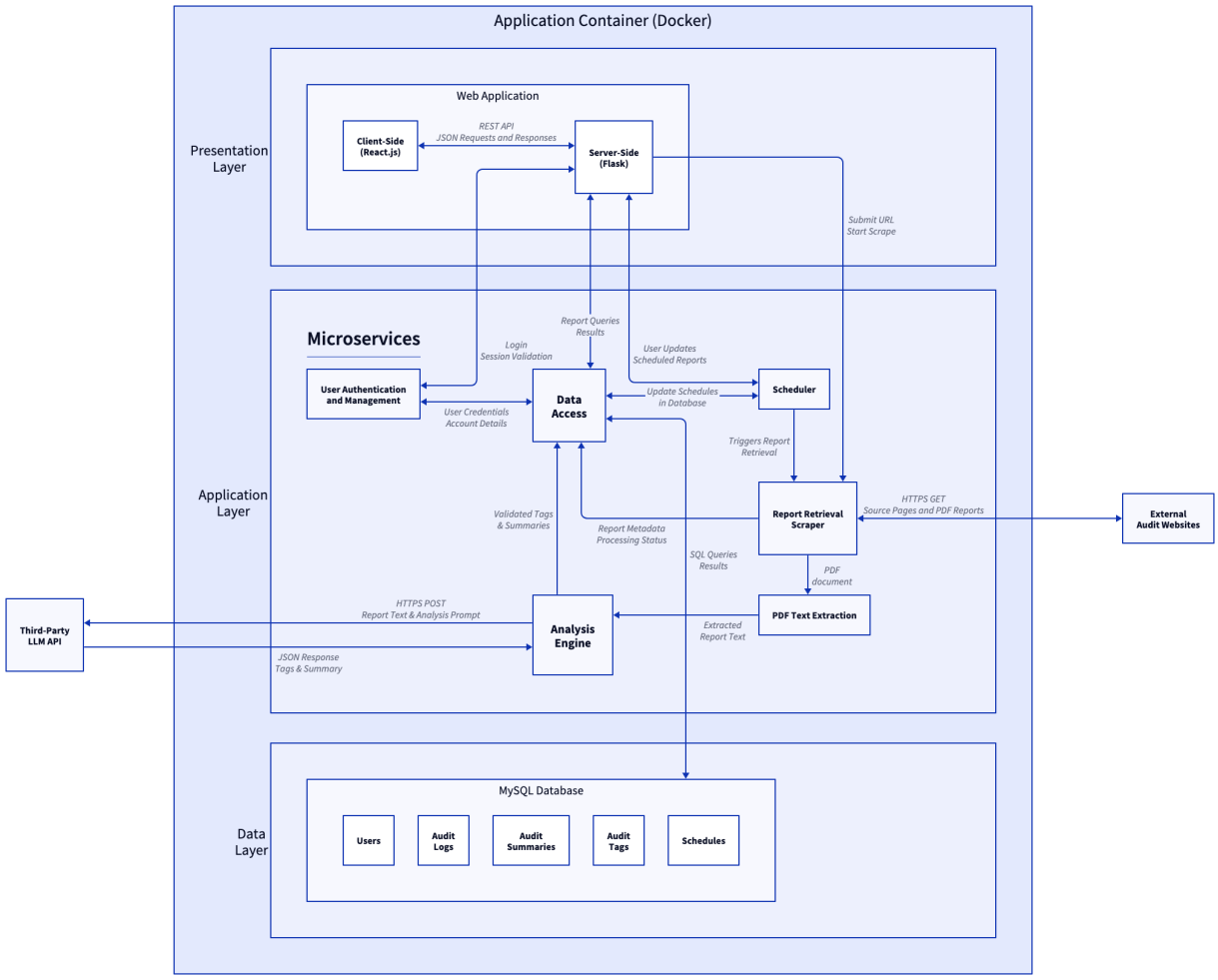
Resource	Usage	Explanation
Node.js (v.26.8 or newer)	Server-Side JavaScript runtime	Required for running server-side JavaScript and development applications like Zustand
TypeScript	Programming language for the client-side presentation layer	TypeScript will be used to build the React-based client-side web application.
HTML	Web client-side markup language	This is required for building pages for the web application.
CSS	Web client-side styling	This is required to format web pages.
React.js	Enable interactivity in client-side web app	This is the standard library used for web application front ends for UI interactivity.
shadcn (v4.21 or newer)	Primary React component library	Provides consistent, interactive, and interoperable components to build a user interface
TanStack Table (v9)	Display tabular data in the UI	This library works with shadcn to display tabular data and charts neatly.
Recharts (v3.10 or newer)	Display charts in the user interface	This library will allow the application to display tabular data and charts neatly.
Tailwind (v4.3 or newer)	Styling web app	CSS library to maintain consistent appearance across the app.
Zustand (v5.0 or newer)	Web application client-side state management	This library makes client-side state management simpler for React applications.
Chrome (v152 or newer)**, Firefox** (v156 or newer)	Web application display and debugging	Users will interact with the solution through a web browser window. Browser developer tools help design and debug a web application.
Flask (v3.1.3)	Web app backend	Flask is a simple Python web application framework, which will route requests between the web app frontend and the application layer.
Python (v3.9 or newer)	Programming language for web app backend	General-purpose programming language used by Flask.
Docker (v29.8 or newer)	App containerization	Using containers will make the solution simpler for clients to deploy on any type of infrastructure.
Git (v2.55 or newer)	Version control	Tracks code changes and allows the development team to work together on a single codebase.

Resource	Usage	Explanation
GitHub	Collaborative version control, project management, application documentation, CI/CD	Team members will push code to GitHub, allowing us to review, integrate, and deploy from a single location.
MySQL (Community Edition)	Database Management System	MySQL will be the managing service for the data layer. It will hold all tables and requests to fetch data will be done via API running a SQL read/write.
External Audit Websites	Source of report information	Provides the publicly available audit documents that the scraper retrieves and processes.
Qwen3.8	Localized LLM running	Open-sourced with Apache 2.0 license. Can be used for experimentation within containers. Will be eventually implemented to strengthen municipal data security.
VT ARC	External LLM API	Primary LLM for pilot product. Will be used for document parsing and tagging. Compatible with common LLM API endpoints, allowing clients to use a different service.
Transformers	Used to manage Qwen model for localized LLM running.	The transformers Python package has a function called 'Pipeline' which will be used to process prompts locally. Can set token size, temperature and use batching to optimize outputs.
BeautifulSoup4	Python package for dynamic web scraping	BeautifulSoup allows Python to request and interact with web pages to find content. It can simulate JavaScript actions to load information hidden behind links, which will be useful for locating PDFs.
FastAPI	Creation of APIs for connecting the 3-layers.	This is a Python package that can be used to easily implement internal APIs. The frontend can use APIs created using this package to manipulate objects in application layer or for application layer to handle data.

3 High-Level Architecture Design

The diagram below illustrates our proposed architecture and the interfaces between component, including:

- The three layers that comprise our overarching architecture.
- The Presentation Layer's client-server architecture and the server-side web application that owns the interface with the Application Layer.
- The Application Layer's microservices architecture and the Data Access service that owns the interface with the Data Layer.
- The Data Layer's implementation as a single database with tables for key Application functions.
- External interfaces with APIs and web pages.



4 Design Sketches

See Attachments section for wireframes.

5 AI Usage Statement

Amr: I used Claude to help me understand some of the design concepts we were working with, like layered architecture, how the LLM piece fits into the system, and how the components connect to each other. All actual design decisions and content came from the team.

6 Attachments

6.1 Wireframes

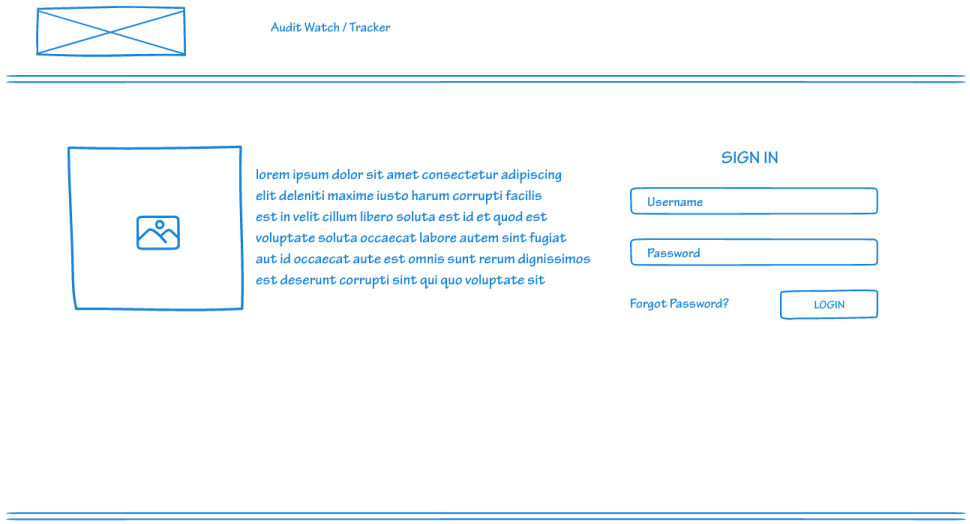


Figure 1: Index / Login

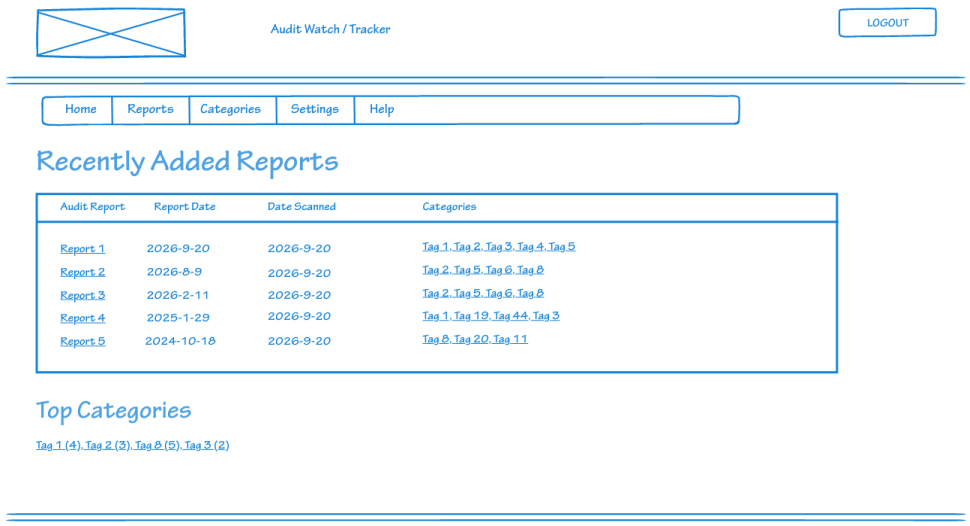


Figure 2: Logged in page

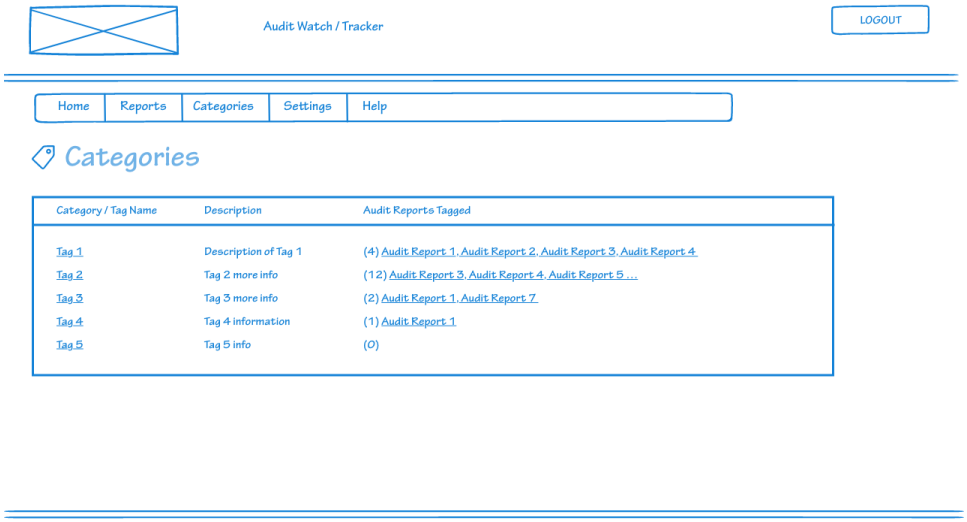


Figure 3: Categories Page

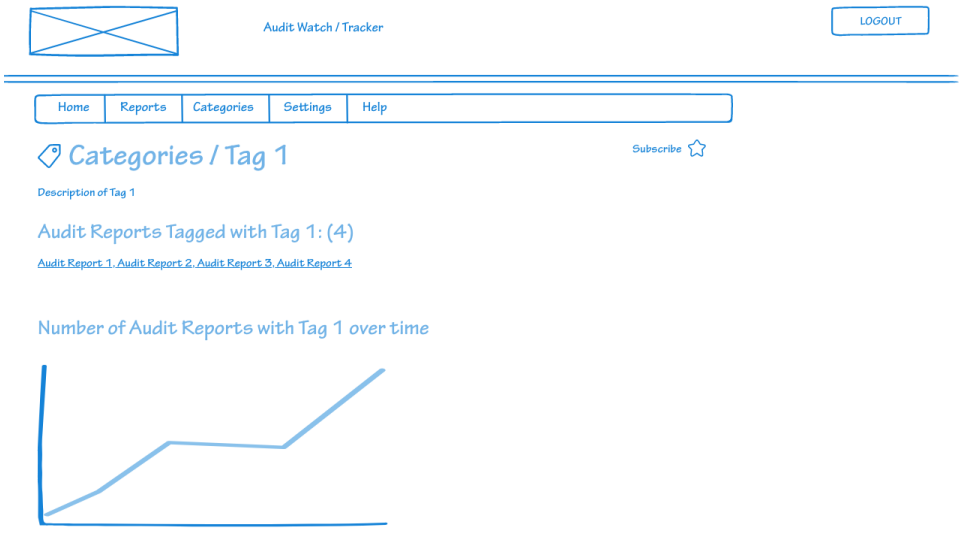


Figure 4: Categories / Tag 1



Figure 5: Categories / Tag 1 / Subscribe

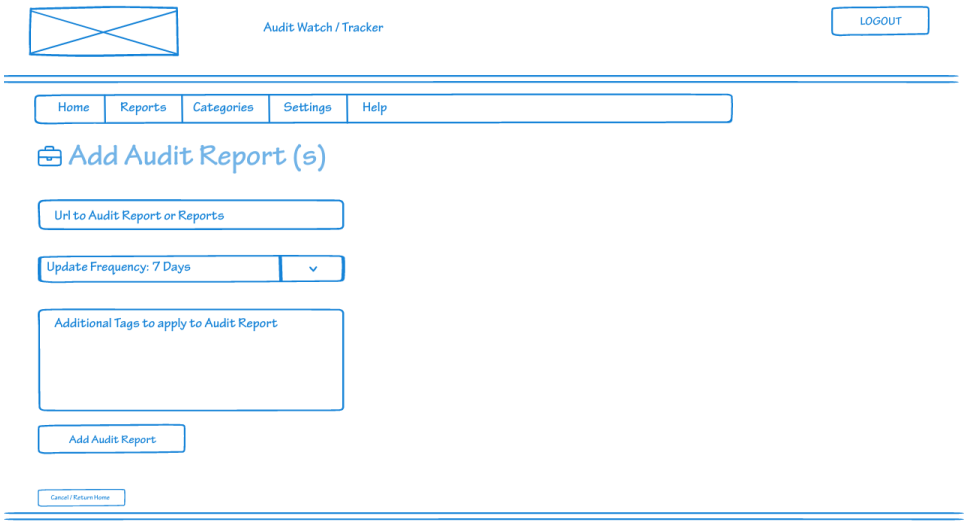


Figure 6: Add Audit Report

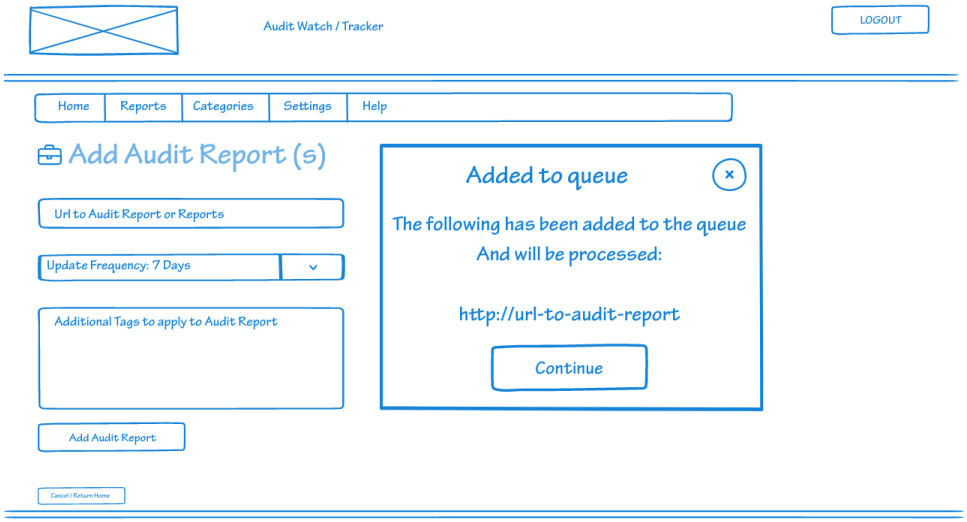


Figure 7: Add Audit Report Alert

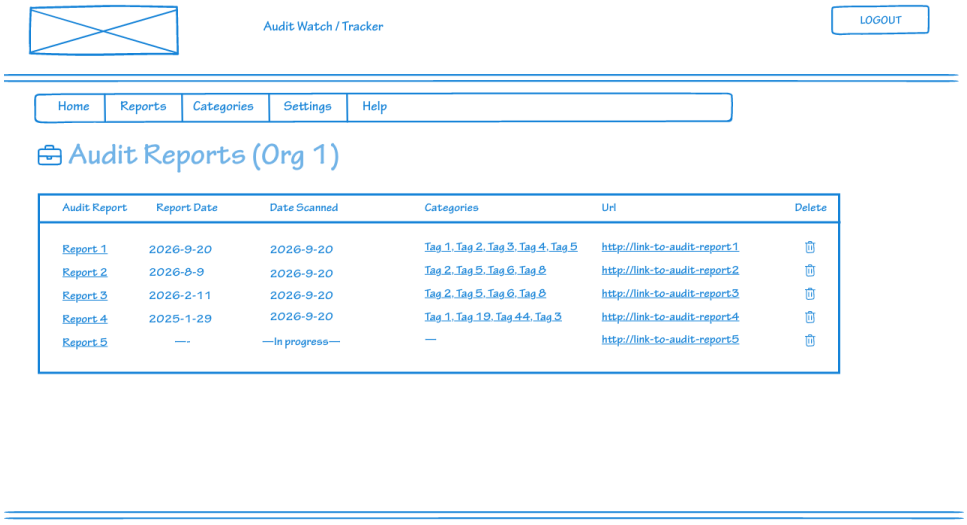


Figure 8: List of audit reports

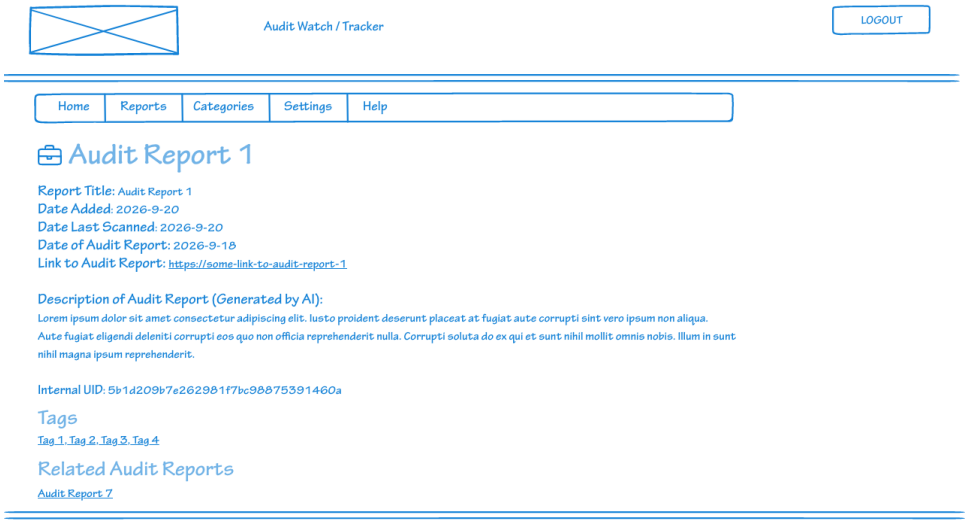


Figure 9: Audit Report 1

6.2 Entity Relationship Diagram

